


# Writing on the Wall

28th January 2024 / Document No. D24.102.188

Prepared By: `w3th4nds`

Challenge Author(s): `w3th4nds`

Difficulty: Very Easy

Classification: Official

# Synopsis

- Writing on the wall is a very easy difficulty challenge that features `off-by-one`, `strcmp null strings`.

# Description

- As you approach a password-protected door, a sense of uncertainty  envelops you—no clues, no hints. Yet, just as confusion takes hold, your gaze locks onto cryptic markings adorning the nearby wall. Could this  be the elusive password, waiting to unveil the door's secrets?

## Skills Required

- Basic C.

## Skills Learned

- `strcmp` stops at null bytes.

# Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'./glibc/'
```

## Protections 🛡️

As we can see:

| Protection | Enabled | Usage |
|---|---|---|
| **Canary** | ✅ | Prevents **Buffer Overflows** |
| **NX** | ✅ | Disables **code execution** on stack |
| **PIE** | ✅ | Randomizes the **base address** of the binary |
| **RelRO** | **Full** | Makes some binary sections **read-only** |

All protections are enabled.

The program's interface

```
〰③ ╤ ℙ Å 𝕀 $̶

The writing on the wall seems unreadable, can you figure it out?

>> aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

[-] You activated the alarm! Troops are coming your way, RUN!
```

There is no signal of Buffer Overflow. We need to take a better look at the program.

# Disassembly

Starting with `main()`:

```
undefined8 main(void)

{
  int iVar1;
```

```
  long in_FS_OFFSET;
  char local_1e [6];
  undefined8 local_18;
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_18 = 0x2073736170743377;
  read(0,local_1e,7);
  iVar1 = strcmp(local_1e,(char *)&local_18);
  if (iVar1 == 0) {
    open_door();
  }
  else {
    error("You activated the alarm! Troops are coming your way, RUN!\n");
  }
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return 0;
}
```

The program is pretty straightforward. It reads our `7-byte` input at `local_1e` and then it compares it with `local_18` which is the string "w3tpass ".

```
>>> import binascii
>>> print(binascii.unhexlify('2073736170743377').decode('utf-8')[::-1])
w3tpass
```

The string is `8-bytes` long but we can only enter 7 bytes, meaning we will never be able to pass the comparison.

> DESCRIPTION
> The strcmp() function compares the two strings s1 and s2. It returns an integer less than, equal to, or greater than zero if s1 is found, respectively, to be less than, to match, or be greater than s2.
>
> The strncmp() function is similar, except it compares only the first (at most) n bytes of s1 and s2.
>
> RETURN VALUE
> The strcmp() and strncmp() functions return an integer less than, equal to, or greater than zero if s1 (or the first n bytes thereof) is found, respectively, to be less than, to match, or be greater than s2.

We need to somehow make this comparison true. Something that is written in the `man` page, is that `strcmp` stops when it reads `\x00` which is the null byte or the terminating character of a string. Having that in mind, we can overflow the buffer and the next thing that will be overwritten, is `local_18` which is the "w3tpass " buffer. If we do so, we will have something like `\x003tpass`. `strcmp` will stop when it reaches the `\x00`, thinking that this is the whole string. What we can do, is filling the buffer with a null byte and junk and then the last byte that will be overwritten to `local_18`, should be null again so that `strcmp` compares 2 empty strings.